IF25-22014 PENGEMBANGAN APLIKASI WEB

# Pertemuan 13
# AI pada Aplikasi Web

Menambahkan fitur model bahasa ke aplikasi dengan aman, terukur, dan jujur kepada pengguna

**Tim Pengajar**
Tahun 2026. Rujukan: Dokumentasi FastAPI, HTTPX, dan Pydantic; dokumentasi API penyedia model yang dipakai (misalnya Ollama untuk model lokal).

ITERA

DIKTISAINTEK BERDAMPAK

**Program Studi Teknik Informatika**
Institut Teknologi Sumatera

PETA PERTEMUAN

# Perjalanan kalian hari ini

ITERA

| Fase | Menit | Yang kalian lakukan | Pertanyaan wajar | Bila tersendat |
|---|---|---|---|---|
| Pembuka | 0–10 | Menjawab tiga soal minggu lalu dengan jari | Masih ingat yang kemarin? | Catat nomor yang salah, cek di modul minggu lalu |
| Coba dulu | 10–30 | Mengerjakan tiga misi berpasangan | Boleh salah? | Tulis dugaan; salah itu bahan belajar |
| Materi | 30–85 | Mengikuti 5 bagian: tebak, ikuti bersama, engsel | Sedang di bagian mana? | Lihat penanda di atas judul atau klik Outline |
| Hands-on | 85–120 | Mengerjakan tingkat 1 sampai 3 dari `starter/` | Sudah benar belum? | Periksa berpasangan; buka modul bagian yang sama |
| Papan | 120–140 | Menunjukkan atau menanggapi hasil teman | Kenapa kode itu jalan? | Ajukan satu pertanyaan mengapa |
| Exit ticket | 140–150 | Menjawab dua soal tanpa catatan | Apa yang belum paham? | Tulis di kolom refleksi |

Penanda fase dan bagian ada di atas setiap judul slide. Kotak **Ikuti bersama** berarti ketik di laptop kalian sekarang.

# Soal dari Pertemuan 12

ITERA

**Soal 1.** Data baru tersimpan permanen setelah?

- A db.add
- B db.commit
- C db.get
- D select

**Soal 2.** Password basis data sebaiknya disimpan di?

- A kode
- B .env yang di-gitignore
- C README
- D JavaScript

**Soal 3.** Menunggu lebih dari 10 detik, antarmuka sebaiknya?

- A diam
- B progres dan tombol batal
- C alert
- D muat ulang

**KUNCI**

1B: commit menutup transaksi. 2B: rahasia tidak ikut ke git. 3B: ambang ketiga dari P1.

# Pengumuman yang terlalu panjang

ITERA

Situs himpunan memuat pengumuman sepanjang dua halaman. Ketua himpunan ingin tombol *Ringkas* yang membuat tiga poin penting dan kategorinya.
Seorang anggota langsung menulis kode di samping, di komponen Next.js, lengkap dengan kunci API penyedia AI.

**PIKIRKAN**

Apa saja yang salah dengan rancangan ini, dari sisi keamanan, biaya, dan pengalaman pengguna?

```
"use client";
const KUNCI = "sk-rahasia-123...";
async function ringkas(teks) {
  const r = await
fetch("https://api.penyedia.ai/v1/chat",
{
    headers: { Authorization: `Bearer
${KUNCI}` },
    method: "POST",
    body: JSON.stringify({ pesan: teks }),
  });
  setHasil(await r.text());
}
```

# Misi kalian, 20 menit

ITERA

Berdua atau bertiga, pakai apa saja yang kalian tahu. Berkas awal ada di folder hands-on, `starter/`.

**1 Rancang**

Tulis apa yang harus dikirim ke model, dan bentuk jawaban yang kalian harapkan agar mudah ditampilkan di halaman React.

**2 Cari kebocoran**

Buka DevTools pada halaman yang memakai kode di masalah minggu ini. Di mana kunci API bisa terlihat?

**3 Coba lokal**

Jalankan `starter/` dalam mode tiruan, lalu panggil `POST /ringkas` dari `/docs`.

**Boleh**

Bertanya ke teman, mencoba di browser, salah.

**Belum dulu**

Mencari di internet atau bertanya ke AI.

# Lima bagian hari ini

ITERA

| | Bagian | Tujuan (klik judul untuk melompat) |
|---|---|---|
| 1 | **AI di aplikasi web** | **Mengenali fitur AI yang umum dan cara aplikasi memakainya** |
| 2 | Memanggil model dari server | Memanggil API model dari FastAPI dengan kunci yang aman |
| 3 | Prompt dan keluaran terstruktur | Menulis prompt yang jelas dan memvalidasi jawaban model dengan Pydantic |
| 4 | Keamanan dan etika | Mengenali risiko fitur AI dan cara mengendalikannya |
| 5 | Antarmuka fitur AI | Merancang antarmuka yang jujur dan nyaman untuk fitur AI |

# Model bahasa sebagai layanan

ITERA

**Large Language Model (LLM)**
Model AI yang dilatih pada teks dalam jumlah sangat besar untuk memprediksi kelanjutan teks; dipakai lewat API untuk meringkas, mengklasifikasi, menjawab, atau mengubah format teks.

Bagi aplikasi web, model adalah satu server lagi di peta P1: kita mengirim request berisi teks, menerima response berisi teks. Bedanya, jawabannya tidak pasti sama setiap kali, bisa keliru dengan yakin, butuh waktu lebih lama, dan berbiaya per pemakaian.

# Fitur AI yang umum

ITERA

| Fitur | Contoh di kampus | Bentuk keluaran |
|---|---|---|
| meringkas | pengumuman panjang menjadi tiga poin | list teks |
| mengklasifikasi | laporan fasilitas ke kategori dan prioritas | satu nilai dari daftar |
| mengekstrak | tanggal dan tempat dari undangan acara | object bertipe |
| menjawab dari dokumen | pertanyaan tentang pedoman akademik | teks dan rujukan |
| menyarankan | mata kuliah pilihan sesuai minat | list dengan alasan |

**Ingat**

Fitur yang paling mudah dipercaya adalah yang keluarannya bisa diperiksa: kategori dari daftar, object dengan kolom jelas.

# Lima bagian hari ini

ITERA

| | Bagian | Tujuan, dan inti bagian yang sudah dibahas |
|---|---|---|
| 1 | AI di aplikasi web | Inti: Aplikasi web memakai model AI lewat API seperti layanan lain: server mengirim teks, model membalas teks yang harus diperiksa. |
| **2** | **Memanggil model dari server** | **Memanggil API model dari FastAPI dengan kunci yang aman** |
| 3 | Prompt dan keluaran terstruktur | Menulis prompt yang jelas dan memvalidasi jawaban model dengan Pydantic |
| 4 | Keamanan dan etika | Mengenali risiko fitur AI dan cara mengendalikannya |
| 5 | Antarmuka fitur AI | Merancang antarmuka yang jujur dan nyaman untuk fitur AI |

# Jangan dari browser

ITERA

## Salah: Next.js langsung

```
// semua kode client terkirim ke browser
const KUNCI = "sk-rahasia-123";
fetch("https://api.penyedia.ai/...", {
  headers: { Authorization:
    `Bearer ${KUNCI}` } });
```

## Benar: lewat FastAPI

```
// Next.js
fetch("http://localhost:8000/ringkas", {
  method: "POST", body: JSON.stringify({ teks
}),
  headers: { "Content-Type": "application/json"
} });

# FastAPI membaca kunci dari .env
KUNCI = os.environ["LLM_API_KEY"]
```

**Bedanya**

Kode komponen klien dikirim utuh ke browser (P5), jadi kunci terlihat di tab Sources atau Network. Siapa pun bisa memakainya atas biaya kalian. Jawaban misi 2.

# Klien LLM di FastAPI

```
import os, httpx

BASE             =             os.environ.get("LLM_BASE_URL",
"http://localhost:11434/v1")
MODEL = os.environ.get("LLM_MODEL", "llama3.2")
KUNCI = os.environ.get("LLM_API_KEY", "")

async def tanya_llm(sistem: str, isi: str) → str:
    async with httpx.AsyncClient(timeout=60) as c:
        r = await c.post(f"{BASE}/chat/completions",
            headers={"Authorization": f"Bearer {KUNCI}"},
            json={"model": MODEL, "messages": [
                {"role": "system", "content": sistem},
                {"role": "user", "content": isi}]})
        r.raise_for_status()
        return r.json()["choices"][0]["message"]["content"]
```

`timeout=60`: jawaban model bisa lama. `raise_for_status`: kode 4xx dan 5xx dari penyedia menjadi error di server kita.

# Ikuti bersama: fitur AI dalam mode tiruan

ITERA

1. Salin konfigurasi contoh; di Windows pakai `copy`.
   ```
   cp .env.example .env
   ```
2. Pastikan modenya tiruan.
   ```
   LLM_MODE=tiruan
   ```
3. Jalankan server.
   ```
   fastapi dev main.py
   ```
4. Di `/docs`, pilih `POST /ringkas`, isi teks minimal 50 karakter, Execute.
5. Ganti ke `LLM_MODE=sungguhan` bila Ollama siap, simpan, ulangi.

**YANG HARUS TERLIHAT**

Mode tiruan membalas dua poin contoh berkategori lainnya. Mode sungguhan membalas ringkasan dari model, atau 503 bila model belum berjalan.

**Kalau berbeda**

Mode tidak berubah: `.env` tidak terbaca; jalankan server dari folder yang sama dengan `.env`. 503 terus: jalankan `ollama serve` lalu `ollama pull llama3.2`.

# Lima bagian hari ini

ITERA

| | Bagian | Tujuan, dan inti bagian yang sudah dibahas |
|---|---|---|
| 1 | AI di aplikasi web | **Inti:** Aplikasi web memakai model AI lewat API seperti layanan lain: server mengirim teks, model membalas teks yang harus diperiksa. |
| 2 | Memanggil model dari server | **Inti:** Model selalu dipanggil dari server; kunci API di .env, alamat dan nama model di konfigurasi, dan pemanggilan memakai klien HTTP asinkron. |
| **3** | **Prompt dan keluaran terstruktur** | **Menulis prompt yang jelas dan memvalidasi jawaban model dengan Pydantic** |
| 4 | Keamanan dan etika | Mengenali risiko fitur AI dan cara mengendalikannya |
| 5 | Antarmuka fitur AI | Merancang antarmuka yang jujur dan nyaman untuk fitur AI |

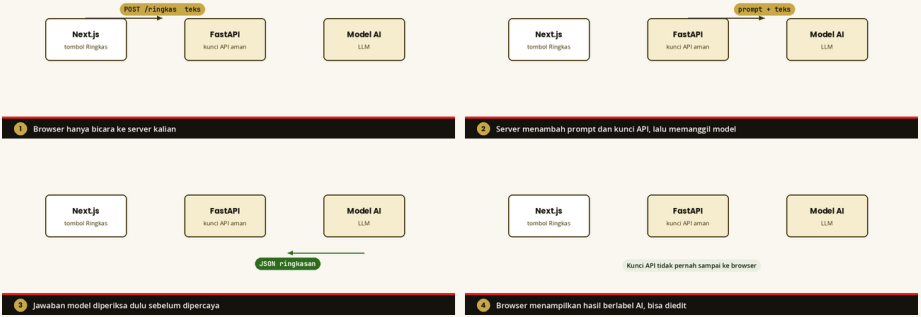
# Minta JSON, lalu periksa

ITERA

```
class Ringkasan(BaseModel):
    poin: list[str] = Field(min_length=1, max_length=3)
    kategori:   Literal["akademik",  "kegiatan",  "beasiswa",
"lainnya"]
SISTEM = """Kamu meringkas pengumuman kampus untuk mahasiswa.
Balas HANYA JSON: {"poin": [maks 3 kalimat], "kategori": salah
satu
dari akademik, kegiatan, beasiswa, lainnya}.  Jangan menambah
fakta."""
@app.post("/ringkas", response_model=Ringkasan)
async def ringkas(m: TeksIn):
    jawaban = await tanya_llm(SISTEM, m.teks)
    try:
        return Ringkasan.model_validate_json(jawaban)
    except ValidationError:
        raise  HTTPException(502,  "Jawaban  AI  tidak  sesuai
format")
```

**502 Bad Gateway**: kesalahan dari layanan yang kita panggil, bukan dari pengguna (4xx) atau kode kita (500).

# Alur fitur AI




Browser tidak pernah melihat kunci API, dan jawaban model diperiksa sebelum sampai ke pengguna.

# Kode status untuk gagal di model

ITERA

Penyedia model membalas 503 karena sibuk, sehingga endpoint `/ringkas` kalian tidak bisa menjawab. Kode status yang paling tepat untuk Next.js?

- **A** 200 dengan ringkasan kosong
- **B** 422
- **C** 502 atau 503
- **D** 404

**JAWABAN: C**

Masalahnya di layanan hulu, bukan di request pengguna. 502 atau 503 memberi tahu frontend untuk menyarankan coba lagi nanti.

# Lima bagian hari ini

ITERA

| | Bagian | Tujuan, dan inti bagian yang sudah dibahas |
|---|---|---|
| 1 | AI di aplikasi web | **Inti:** Aplikasi web memakai model AI lewat API seperti layanan lain: server mengirim teks, model membalas teks yang harus diperiksa. |
| 2 | Memanggil model dari server | **Inti:** Model selalu dipanggil dari server; kunci API di .env, alamat dan nama model di konfigurasi, dan pemanggilan memakai klien HTTP asinkron. |
| 3 | Prompt dan keluaran terstruktur | **Inti:** Prompt yang baik menyebut peran, tugas, batasan, dan format; minta JSON lalu validasi dengan Pydantic, dan balas 502 bila jawaban model tidak sesuai. |
| **4** | **Keamanan dan etika** | **Mengenali risiko fitur AI dan cara mengendalikannya** |
| 5 | Antarmuka fitur AI | Merancang antarmuka yang jujur dan nyaman untuk fitur AI |

# Risiko dan pengendaliannya

ITERA

| Risiko | Pengendalian |
|---|---|
| kunci bocor, tagihan membengkak | kunci hanya di server dan `.env`; batasi jumlah permintaan per pengguna, balas 429 bila lewat |
| prompt injection | teks pengguna bisa berisi *abaikan aturan sebelumnya*; jangan beri model wewenang langsung, validasi keluaran |
| data pribadi | jangan kirim NIM, nilai, atau kontak bila tidak perlu; pahami kebijakan data penyedia |
| halusinasi | jawaban keliru dengan yakin; minta model tidak menambah fakta, tampilkan sumber, biarkan manusia memeriksa |
| bias dan ketidakadilan | uji dengan contoh beragam; keputusan penting tetap di tangan manusia |

# Tiga baris yang salah

ITERA

| Yang ditulis | Mengapa salah, dan perbaikannya |
|---|---|
| kunci API di komponen `"use client"` | Terkirim ke setiap browser. Pindahkan pemanggilan ke FastAPI dan kunci ke `.env`. |
| `setHasil(await r.text())` langsung dari model | Jawaban tidak diperiksa; bisa bukan JSON atau berisi teks aneh. Validasi dengan model Pydantic di server. |
| tombol Ringkas tanpa batas | Satu orang bisa menekan ratusan kali atas biaya kalian. Matikan tombol saat memuat dan batasi di server. |

**Pola kesalahannya**

Ketiganya ada di kode masalah minggu ini.

# Lima bagian hari ini

ITERA

| | Bagian | Tujuan, dan inti bagian yang sudah dibahas |
|---|---|---|
| 1 | AI di aplikasi web | **Inti:** Aplikasi web memakai model AI lewat API seperti layanan lain: server mengirim teks, model membalas teks yang harus diperiksa. |
| 2 | Memanggil model dari server | **Inti:** Model selalu dipanggil dari server; kunci API di .env, alamat dan nama model di konfigurasi, dan pemanggilan memakai klien HTTP asinkron. |
| 3 | Prompt dan keluaran terstruktur | **Inti:** Prompt yang baik menyebut peran, tugas, batasan, dan format; minta JSON lalu validasi dengan Pydantic, dan balas 502 bila jawaban model tidak sesuai. |
| 4 | Keamanan dan etika | **Inti:** Lindungi kunci dan biaya, anggap teks pengguna bisa berisi perintah, jangan kirim data pribadi tanpa perlu, dan beri tahu pengguna bahwa hasil AI bisa salah. |
| **5** | **Antarmuka fitur AI** | **Merancang antarmuka yang jujur dan nyaman untuk fitur AI** |

# Pedoman antarmuka fitur AI

ITERA

| Pedoman | Wujudnya di React |
|---|---|
| tunjukkan sedang bekerja dan berapa lama biasanya | teks status *Meringkas, biasanya 5 sampai 15 detik* dengan `role="status"` |
| beri jalan keluar | tombol *Batal* yang menghentikan fetch dengan `AbortController` |
| beri label yang jujur | label *Dibuat AI, periksa sebelum dipakai* di atas hasil |
| biarkan pengguna mengoreksi | hasil ditampilkan di `<textarea>` terkendali yang bisa diedit sebelum disimpan |
| gagal dengan sopan | pesan `role="alert"`: apa yang gagal dan sarankan coba lagi, untuk 502 dan 503 |
| kumpulkan umpan balik | tombol *berguna* dan *tidak berguna* di bawah hasil |

# Batal dan label di Next.js

ITERA

```
const [memuat, setMemuat] = useState(false);
const kendali = useRef(null);
async function ringkas() {
  kendali.current = new AbortController();
  setMemuat(true);
  try {
    const r = await fetch(`${API}/ringkas`, { method: "POST",
      headers: { "Content-Type": "application/json" },
      body:         JSON.stringify({     teks     }),      signal:
kendali.current.signal });
    if (!r.ok) throw new Error(r.status);
    setHasil(await r.json());
  } catch (e) {
    if (e.name !== "AbortError") setGalat("Gagal meringkas.  Coba
lagi.");
  } finally { setMemuat(false); }
} // Batal: kendali.current?.abort()
```

Setelah hasil datang, tampilkan label AI dan teks yang bisa diedit.

# Giliran kalian

ITERA

**1** **Setengah jadi: klasifikasi laporan**

Lengkapi model: `prioritas: _____["rendah", "sedang", "tinggi"]` dan di endpoint, bila validasi gagal, `raise HTTPException(_____)`.

**2** **Mandiri: batas permintaan**

Tulis dependency sederhana yang menyimpan waktu permintaan per alamat IP di dict dan membalas 429 bila lebih dari 5 permintaan dalam 60 detik.

**JAWABAN**

Setengah jadi: Literal, 502. Mandiri: dict IP ke list waktu; buang yang lebih tua dari 60 detik; bila panjangnya 5 atau lebih, raise HTTPException(429); bila tidak, tambahkan waktu sekarang.

# Hands-on bertingkat

ITERA

| Tingkat | Waktu | Yang dilatih | Sub-CPMK |
|---|---|---|---|
| Tingkat 1, Dasar | 10′ | endpoint dengan model masukan dan keluaran dalam mode tiruan | 13.2 |
| Tingkat 2, Menengah | 15′ | memanggil model sungguhan, prompt sistem, validasi JSON, dan 502 | 13.1, 13.2 |
| Tingkat 3, Tantangan | 10′ | antarmuka Next.js: memuat, batal, label AI, edit, galat | 13.2 |
| Pengayaan | sisa | batas permintaan dan umpan balik pengguna | 13.1 |

**Aturan**

Berkas awal ada di `02 Hands-on/P13 - AI - Hands-on/starter`. Kerjakan berurutan dari tingkat 1, simpan bukti tangkapan layar, dan sebutkan alasan setiap langkah.

# Presentasi di papan

ITERA

Tiga mahasiswa diundi untuk menunjukkan hasil hands-on tingkat 2 atau 3 di layar, masing-masing sekitar 5 menit, lalu menjawab satu pertanyaan.

| Kriteria | Sangat baik 75--100 | Baik 65--74 | Cukup 55--64 | Kurang <55 |
|---|---|---|---|---|
| Ketepatan hasil | berjalan dan sesuai soal | satu salah kecil | ada bagian salah | sebagian besar keliru |
| Alasan tiap langkah | setiap baris dijelaskan fungsinya | sebagian besar dijelaskan | penjelasan kabur | tanpa penjelasan |
| Menjawab pertanyaan | tanpa melihat catatan | dengan sedikit bantuan | menjawab sebagian | tidak bisa menjawab |

Yang tidak maju tetap bisa menambah nilai lewat pertanyaan atau koreksi yang tepat.

# Exit ticket Pertemuan 13

ITERA

```
@app.post("/klasifikasi")
async def klasifikasi(m: LaporanIn):
    return await tanya_llm(SISTEM, m.teks)
```

**1 Sub-CPMK 13.1**

Sebutkan dua alasan mengapa model AI dipanggil dari FastAPI, bukan langsung dari komponen Next.js.

**2 Sub-CPMK 13.2**

Endpoint di atas mengembalikan teks model apa adanya. Tulis model Pydantic untuk keluaran berisi `kategori` dari $3 + (A \bmod 2)$ pilihan dan perbaikan endpoint-nya, termasuk kode status bila jawaban tidak sesuai.

**3 Refleksi**

Tulis satu kalimat: bagian mana yang paling membingungkan hari ini, dan mengapa?

Soal 1 dan 2 dinilai 0, 1, atau 2: benar dengan alasan, benar tanpa alasan atau salah kecil, belum benar.

# Yang dibawa pulang

ITERA

| | Bagian | Inti |
|---|---|---|
| 1 | AI di aplikasi web | Aplikasi web memakai model AI lewat API seperti layanan lain: server mengirim teks, model membalas teks yang harus diperiksa. |
| 2 | Memanggil model dari server | Model selalu dipanggil dari server; kunci API di .env, alamat dan nama model di konfigurasi, dan pemanggilan memakai klien HTTP asinkron. |
| 3 | Prompt dan keluaran terstruktur | Prompt yang baik menyebut peran, tugas, batasan, dan format; minta JSON lalu validasi dengan Pydantic, dan balas 502 bila jawaban model tidak sesuai. |
| 4 | Keamanan dan etika | Lindungi kunci dan biaya, anggap teks pengguna bisa berisi perintah, jangan kirim data pribadi tanpa perlu, dan beri tahu pengguna bahwa hasil AI bisa salah. |
| 5 | Antarmuka fitur AI | Tampilkan perkiraan waktu, sediakan batal, beri label AI, dan biarkan hasil diedit. |

**Sebelum Pertemuan 14:** kerjakan Cek pemahaman Modul 13, lalu siapkan demo kemajuan proyek sisi server untuk Pertemuan 14 sesuai Panduan Proyek.




# Ada pertanyaan?

Sampai jumpa di Pertemuan 14: Proyek: Iterasi Sisi Server